1. Introduction

To develop **Java Servlets and Web Applications**, we need two main tools:

1. **Eclipse IDE for Enterprise Java and Web Developers**
2. **Apache Tomcat Server**

Eclipse is used to **write and manage our Java web application**, while Tomcat is used to **run the web application on a server**.

---

2. Download Eclipse IDE

For Servlet and Web Development, download:

Eclipse IDE for Enterprise Java and Web Developers

This version is recommended because it already contains tools needed for:

- Java Web Development
- Servlets
- JSP
- XML
- Maven
- HTML/CSS
- Web projects

Eclipse can be downloaded as a **ZIP file**, so a traditional installation is not always required.

Extract Eclipse

Create a folder such as:

`Softwares`

Extract the Eclipse ZIP file inside this folder.

To start Eclipse, open the extracted Eclipse folder and run the **Eclipse executable**.

When Eclipse starts, it will ask for a **Workspace location**.

A workspace is the folder where Eclipse stores your projects.

---

3. Download Apache Tomcat

Apache Tomcat is a **web server and Servlet container**.

It is used to run Java web applications such as:

- Servlets
- JSP applications
- Other Java web applications

Download the appropriate **Tomcat ZIP file** for your operating system.

For example:

Apache Tomcat 10.1

Extract the downloaded ZIP file into your `Softwares` folder.

A Tomcat folder contains important directories such as:

`bin`

Contains files used to start and stop Tomcat.

`conf`

Contains configuration files.

For example, server ports and users can be configured here.

`webapps`

This is where web applications can be deployed.

---

4. Configure Tomcat in Eclipse

After installing/extracting Eclipse and Tomcat, we need to tell Eclipse where Tomcat is located.

Open Eclipse.

Go to:

Window → Preferences

Then:

Server → Runtime Environments

Click:

Add

Select:

Apache → Apache Tomcat

Choose the Tomcat version you downloaded.

For example:

`Apache Tomcat v10.1`

Click **Next**.

Now click **Browse** and select the **main Tomcat folder**.

Important

Select the Tomcat folder itself.

Do **NOT** select:

`Tomcat/bin`

For example:

`D:\Softwares\apache-tomcat-10.1.x`

Then click:

Finish → Apply and Close

Now Eclipse knows where your Tomcat server is located.

---

5. Open the Servers View

The **Servers View** allows us to manage Tomcat directly from Eclipse.

Go to:

Window → Show View → Other...

Search for:

`Servers`

Then select:

Server → Servers

Click **Open**.

The **Servers** tab will appear, usually at the bottom of Eclipse.

---

6. Add Tomcat Server to Eclipse

Inside the Servers view, you may see:

> No servers are available. Click this link to create a new server.

Click that link.

Select your Tomcat version.

For example:

Apache Tomcat v10.1 Server

Eclipse should automatically detect the Tomcat Runtime Environment that we configured earlier.

Click:

Finish

Now Tomcat will appear in the **Servers** view.

---

7. Start Tomcat

Right-click the Tomcat server.

Select:

Start

Eclipse will start the Tomcat server.

Check the **Console** for messages.

If Tomcat starts successfully without errors, your server configuration is working correctly.

---

8. Create a Dynamic Web Project

Now we can create our first Java Web Project.

Go to:

File → New → Other...

Search for:

`Dynamic Web Project`

Select:

Web → Dynamic Web Project

Click **Next**.

Enter a project name.

For example:

`FirstDemo`

Make sure the **Target Runtime** is your Tomcat server.

For example:

`Apache Tomcat v10.1`

Click **Next** through the configuration screens.

On the final screen, you may see:

Generate web.xml deployment descriptor

You can select this option if you want Eclipse to create the `web.xml` file.

Then click:

Finish

---

9. Create an HTML File

After creating the project, it will appear in the **Project Explorer**.

Expand your project.

Find the web application folder, usually:

`src/main/webapp`

or, depending on the Eclipse/project configuration:

`WebContent`

Right-click the web application folder.

Select:

New → HTML File

Name the file:

`index.html`

Then write your HTML code.

Example:

```
<!DOCTYPE html>
<html>
<head>
    <title>First Demo</title>
</head>
<body>

    <h2>First Java Web Program</h2>

</body>
</html>
```

Save the file.

---

10. Run the Web Application

Right-click `index.html`.

Select:

Run As → Run on Server

Select your Tomcat server.

Click:

Finish

Eclipse will start/deploy the application on Tomcat.

The HTML page should then open in your browser.

---

11. Common Problem: "Selection is not within a valid module"

Sometimes Eclipse may show an error like:

> The selection is not within a valid module.

This usually means Eclipse is not correctly recognizing the project as a web module.

Solution

Right-click your project.

Go to:

Properties → Project Facets

Look for:

Dynamic Web Module

Check that it is enabled.

If necessary, change the Dynamic Web Module version to a compatible version, for example:

`5.0`

Click:

Apply and Close

Then try:

Run As → Run on Server

again.

---

12. Important Terms to Remember

Eclipse IDE

A software development environment where we write and manage Java code and web projects.

Apache Tomcat

A server used to run Java Servlet and JSP applications.

Server Runtime Environment

It tells Eclipse where the Tomcat server is installed and which server version the project should use.

Servers View

An Eclipse window used to start, stop, restart, and manage servers such as Tomcat.

Dynamic Web Project

An Eclipse project designed for creating Java-based web applications.

`webapps`

A Tomcat folder where web applications can be deployed.

`web.xml`

A deployment descriptor used to configure a Java web application. Modern Servlet applications can often use annotations instead, but `web.xml` is still useful to understand.

---

13. Complete Setup Flow

Remember the whole process like this:

Download Eclipse

↓

Download Tomcat

↓

Extract both

↓

Configure Tomcat Runtime in Eclipse

↓

Open Servers View

↓

Add Tomcat Server

↓

Start Tomcat

↓

Create Dynamic Web Project

↓

Create `index.html`

↓

Run on Server

↓

Browser displays the web page

Simple Concept

Think of it this way:

Eclipse = Where we write the application

Tomcat = Where we run the application

Servlet = Java code that handles web requests

Browser = Client that sends requests

So the basic flow is:

Browser → Tomcat → Servlet → Response → Browser

This is the basic foundation you need before moving into actual **Servlet programming**.

Video Link: https://youtu.be/bHZkuwMdLmc?si=7gv3AWfnoxORggeO